CYBERSECURITY TECHNICAL ASSESSMENT REPORT

A Practical Evaluation of Firmware Authenticity, Integrity and Update Security

Secure Firmware Update & Cybersecurity Conformance Assessment

Independent Cybersecurity Project

Reference Framework: ETSI EN 303 645

Emmanuel Antwi Nyamaah
November 2026

Contents

Secure Firmware Update & Cybersecurity Conformance Assessment 0
Contents 1
Table of figures 1
SETUP 2
Firmware Security Test Cases 4
Unsigned firmware file 6
Conceptual Assessment 7
Functional Completeness Assessment (FCA) 9
Functional Sufficiency Assessment (FSA) 11
Overall Findings and Recommendations 12
Conclusion 13
References 14

Table of figures

Figure 1; SETUP-1: Python Environment and Dependency Installation 2
Figure 2; SETUP-2: PowerShell Execution Policy Error 3
Figure 3; SETUP-3: Virtual Environment Activation and Troubleshooting 3
Figure 4; SETUP-4: Firmware Lab Initialisation Verification 3
Figure 5; FW-1: Successful Firmware Update Installation and Verification 4
Figure 6; FW-1.1: Signed Firmware Update Package Verification 4
Figure 7; FW-2: Signed Firmware Version 3 Creation 4
Figure 8; FW-2.1: Original Firmware Before Tampering 4
Figure 9; FW-2.2: Modified Firmware After Tampering 5
Figure 10; FW-2.3: Tampered Firmware Signature Rejection 5
Figure 11; FW-3: Firmware Downgrade Protection 5
Figure 12; Unsigned Firmware 6
Figure 13; FW-4 Unsigned Firmware Rejection 6
Figure 14; CA-1: Cryptographic Key Generation 7
Figure 15; CA-1.1: Firmware Digital Signing 7
Figure 16; CA-1.2: Firmware Signature Verification 8
Figure 17; FCA-1: Cryptographic Key Generation 9
Figure 18; FCA-2: Firmware Digital Signing 9
Figure 19; FCA-3: Digital Signature Verification 9
Figure 20; FCA-4: Firmware Integrity Protection 9
Figure 21; FCA-05: Firmware Downgrade Protection 10
Figure 22; FCA-6: Invalid Signature Handling 10

SETUP

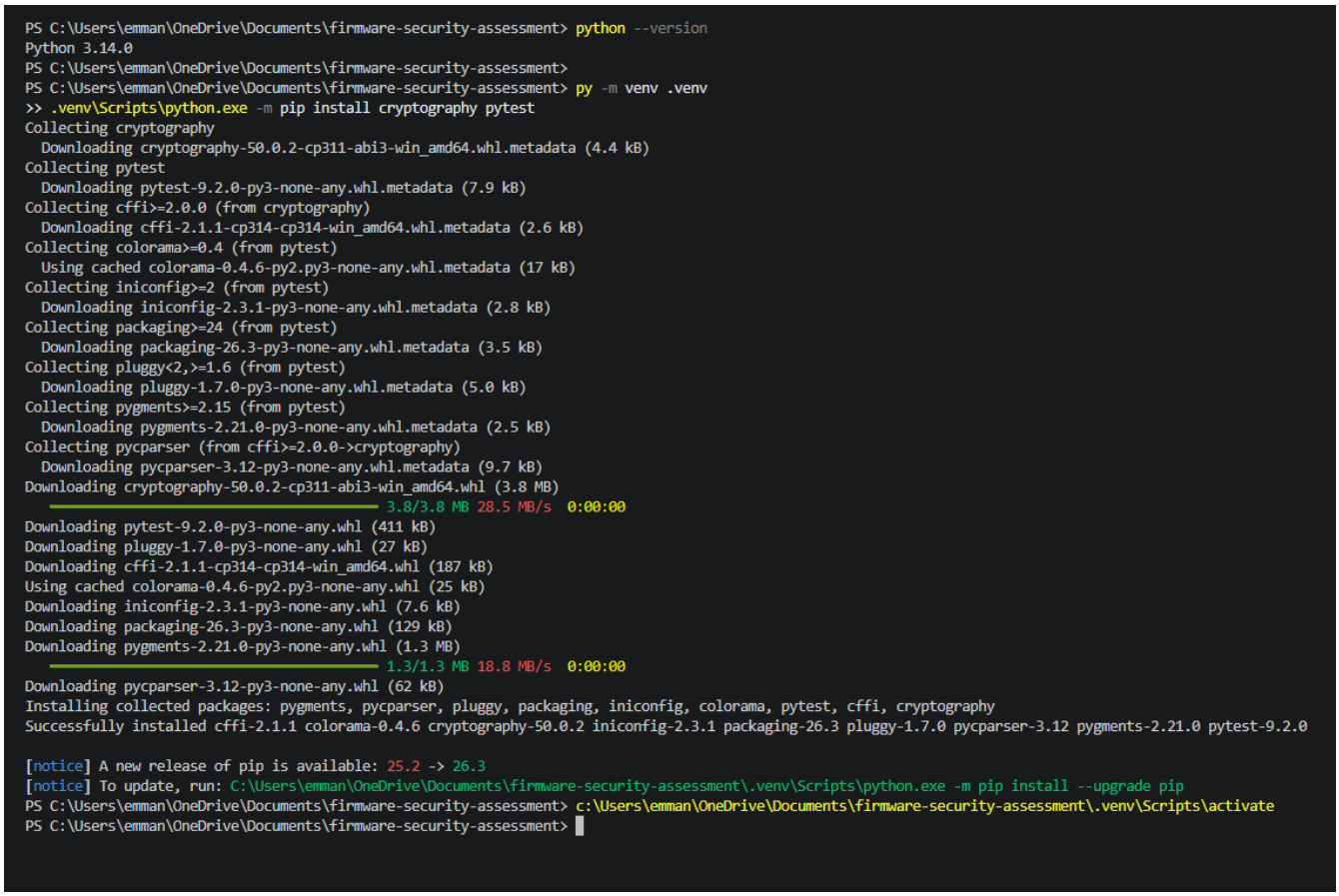


Figure 1; SETUP-1: Python Environment and Dependency Installation

- **Objective:** I aimed to set up my Python development environment and install the necessary libraries for my firmware security assessment project.
- **Python verification:** I ran python --version to verify my Python installation. As shown in the screenshot, I successfully confirmed that Python 3.14.0 was installed.
- **Virtual environment creation:** I executed py -m venv .venv to create an isolated Python environment, allowing me to manage my project's dependencies separately.
- **Dependency installation:** I installed two essential Python libraries:
    - Cryptography: To implement digital signatures and verify firmware authenticity and integrity.
    - Pytest: To automate security testing and validate the functionality of my firmware simulator.
- **Visible evidence:** The screenshot shows the libraries and their dependencies being downloaded and installed. The terminal displays the message "Successfully installed," confirming that the installation completed successfully.

- **Additional observation:** I noticed a notification indicating that a newer version of pip was available. However, this did not affect the successful installation of my dependencies.
- **Conclusion:** I successfully configured the initial Python development environment and installed the required libraries, preparing my system for the development and security assessment of my firmware update simulator.

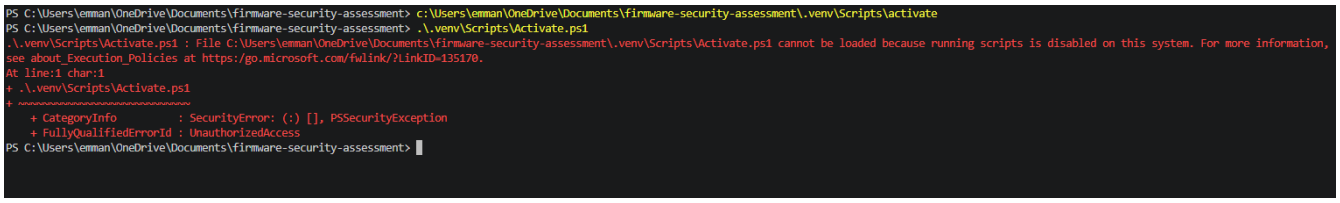


Figure 2; SETUP-2: PowerShell Execution Policy Error

- I attempted to activate my Python virtual environment in VS Code using '\.venv\Scripts\Activate.ps1', but PowerShell blocked the script because script execution is disabled, raising a 'PSSecurityException'. This confirmed that the execution policy was preventing activation. I planned to temporarily adjust the policy for the current session without changing the system-wide configuration.

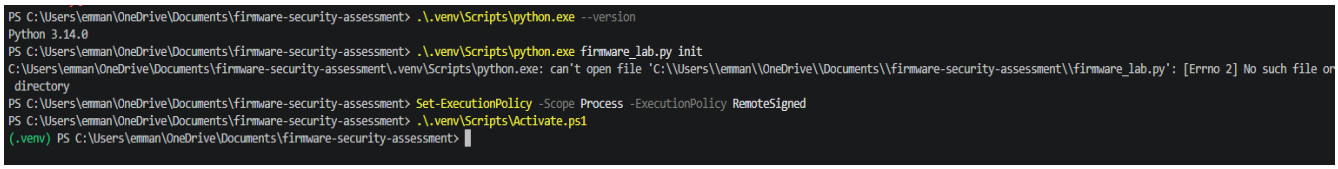


Figure 3; SETUP-3: Virtual Environment Activation and Troubleshooting

- I resolved the PowerShell execution policy issue by temporarily setting the policy to RemoteSigned for the current session, then activated my Python virtual environment. Python 3.14.0 was verified, but the 'firmware_lab.py' script was missing from the project directory and must be located or created before proceeding.

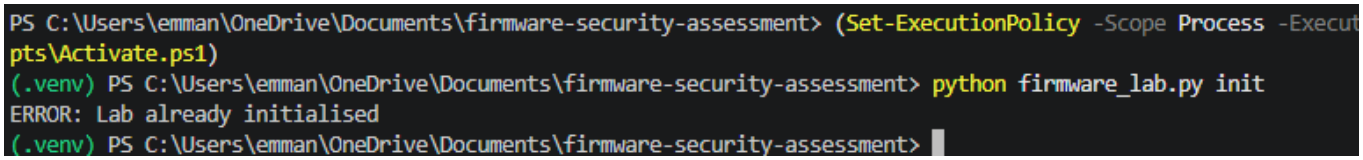


Figure 4; SETUP-4: Firmware Lab Initialisation Verification

- I ran 'python firmware_lab.py init' in the VS Code PowerShell terminal with my virtual environment active. The simulator reported that the lab was already initialised, confirming it detected existing configuration files and ran the command successfully. I planned to check the simulated device's firmware status before security testing.

Firmware Security Test Cases

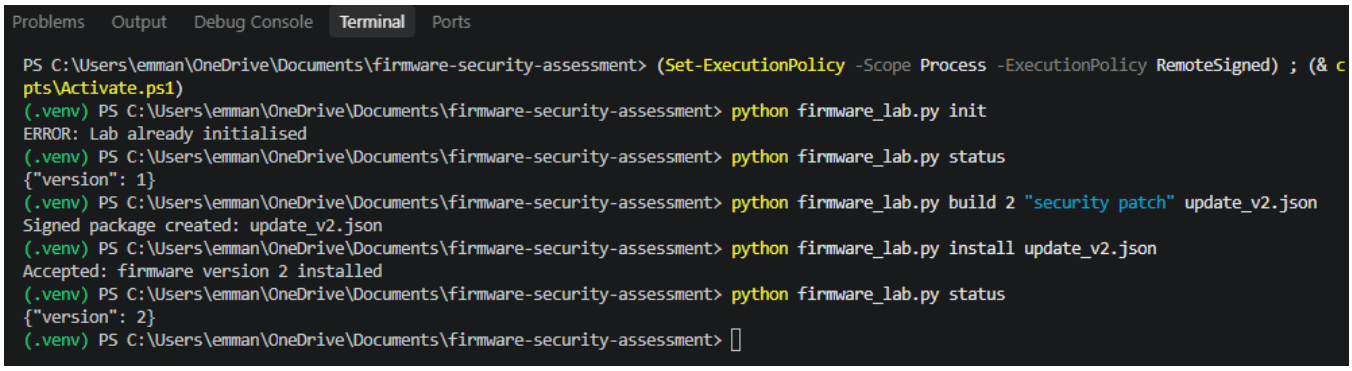


Figure 5; FW-1: Successful Firmware Update Installation and Verification

- I verified that the simulator started at firmware version 1, generated a digitally signed version 2 update containing the “security patch” payload, and successfully installed it. A subsequent status check confirmed the device was running version 2. The test passed, demonstrating the update mechanism's functionality under the tested conditions; further testing is needed to assess its broader security sufficiency.



Figure 6; FW-1.1: Signed Firmware Update Package Verification

- I inspected ‘update_v2.json’ in VS Code and confirmed it contained the firmware version, the “security patch” payload, and a digital signature. These required fields indicate the package includes the expected information for signature verification, but the screenshot alone does not confirm the signature is valid.

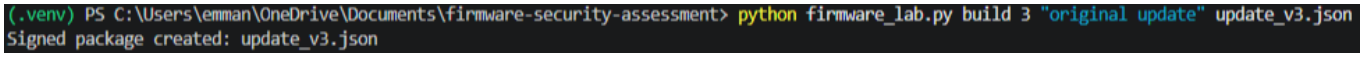


Figure 7; FW-2: Signed Firmware Version 3 Creation

- I ran ‘python firmware_lab.py build 3 "original update" update_v3.json’ in the VS Code terminal and successfully created a signed version 3 firmware package. This confirmed that the simulator can generate signed packages for testing whether it detects unauthorised modifications.

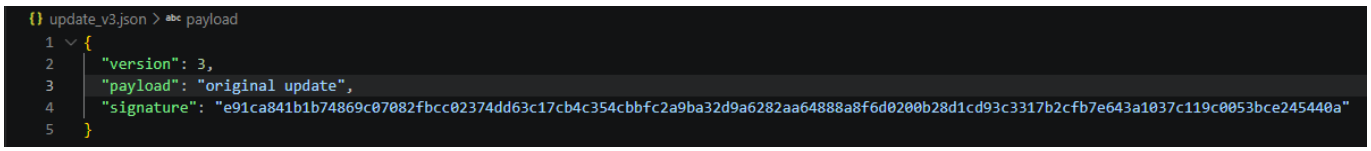


Figure 8; FW-2.1: Original Firmware Before Tampering

- I inspected update_v3.json in VS Code before modifying it for security testing. The package contained the expected version, payload, and digital signature fields, providing a baseline for the planned tampering test.



Figure 9; FW-2.2: Modified Firmware After Tampering

- I modified the payload in update_v3.json from "original update" to "malicious update" while leaving the version and digital signature unchanged. This prepared a tampered firmware package to test whether the simulator rejects updates with modified content; I will verify rejection in the next test.

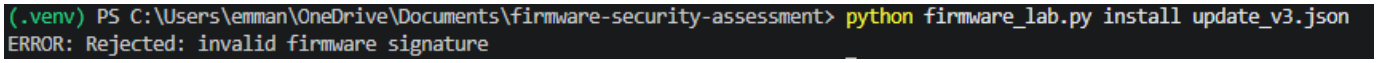


Figure 10; FW-2.3: Tampered Firmware Signature Rejection

- I tested whether the simulator could detect and reject modified firmware by running 'python firmware_lab.py install update_v3.json'. It rejected the update with "ERROR: Rejected: invalid firmware signature," confirming that signature verification prevented installation under the tested conditions.

Report Field	Description
Test ID	FW-02
Security Objective	Firmware integrity and authenticity
Assessment type	Functional sufficiency
Test Method	Modify a signed firmware package and attempt installation
Expected Result	Firmware Rejected
Actual Result	Firmware rejected. Invalid firmware signature
Evidence	Screenshot and terminal output

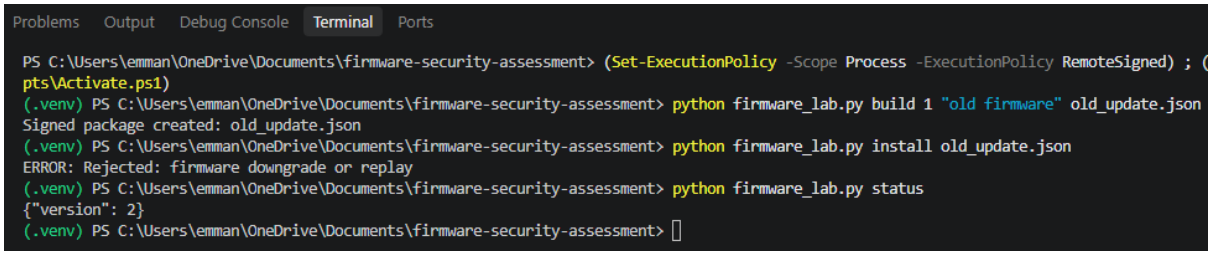


Figure 11; FW-3: Firmware Downgrade Protection

- The functional sufficiency assessment passed: after version 2 was installed, the simulator rejected a signed version 1 firmware package with the error "Rejected: firmware downgrade or replay," and the device's recorded version remained at 2
- **Conclusion:** I confirmed that the simulator's version-checking mechanism prevented a firmware downgrade under the tested conditions.

Unsigned firmware file

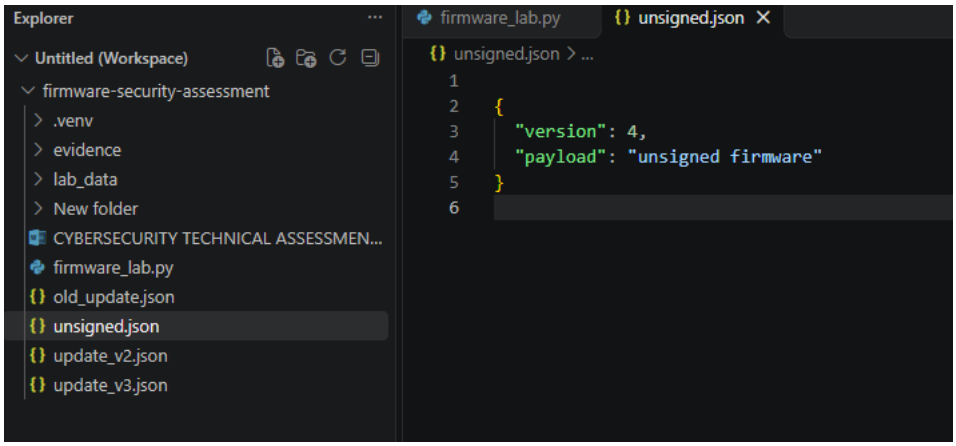


Figure 12; Unsigned Firmware

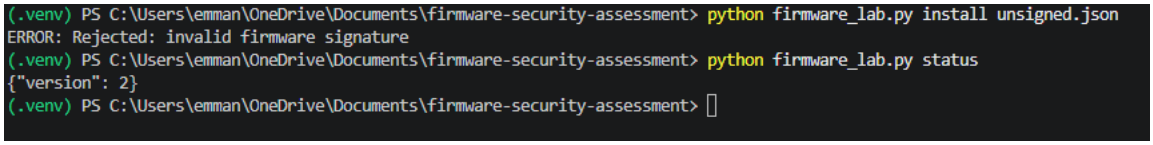


Figure 13; FW-4 Unsigned Firmware Rejection

- I tested the firmware simulator's ability to reject unsigned updates by creating a version 4 package without a digital signature and attempting to install it. The screenshots showed the missing signature field, the terminal's rejection message and the device's unchanged firmware version. The Functional Sufficiency Assessment passed, confirming that unsigned firmware could not be installed under the tested conditions and demonstrating a basic signature requirement, although clearer error handling for missing signatures would improve the simulator.

Field	Description
Objective	Verify rejection of unassigned firmware
Test Method	Attempted installation without a signature
Expected Result	Firmware rejected
Actual Result	Invalid firmware signature; version remained 2
Verdict	Pending verification

Conceptual Assessment

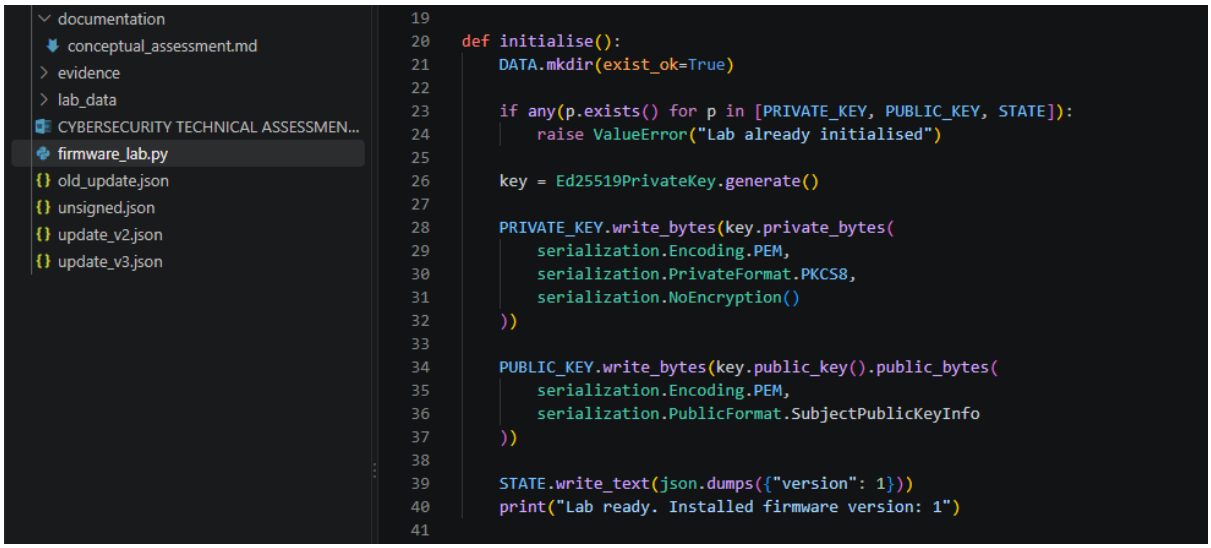


Figure 14; CA-1: Cryptographic Key Generation

In this screenshot, I reviewed the initialise() function, which generates an Ed25519 key pair. The private key signs firmware updates, while the public key verifies their authenticity. I also noted that the private key is stored unencrypted, which could pose a security risk in a real-world system.



Figure 15; CA-1.1: Firmware Digital Signing

In this screenshot, I examined the build() function to understand how firmware updates are digitally signed. The code combines the firmware version and payload, then uses the private key to create a digital signature. This allows verification to detect any changes to the signed firmware information.

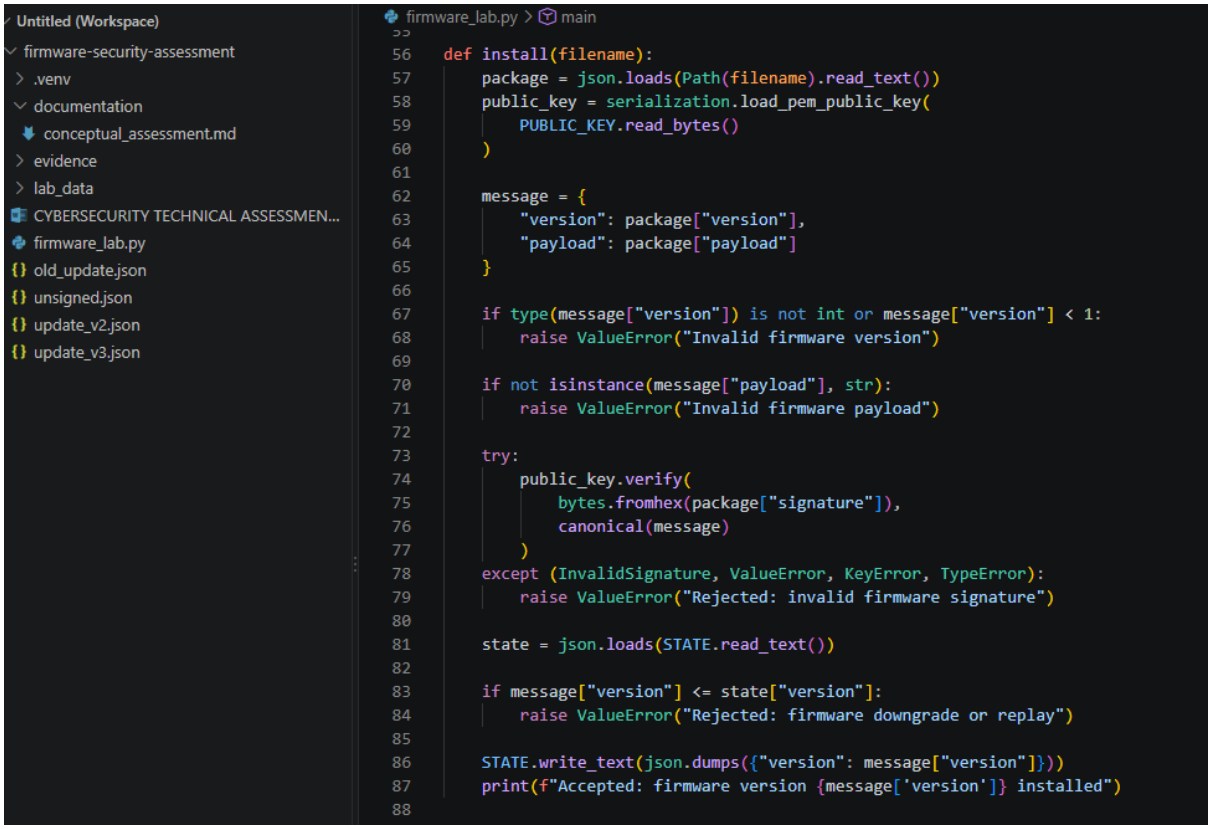


Figure 16; CA-1.2: Firmware Signature Verification

The screenshot shows install() verifying firmware signatures with a trusted public key and checking versions to block downgrades or duplicate installations. This prevents tampered, unauthorised, or outdated packages from updating the recorded firmware version.

In my conceptual assessment, I found a limitation in the firmware update mechanism. While the install() function verifies digital signatures and checks firmware versions, it does not install a firmware binary. Instead, line 86 updates device_state.json to record the new firmware version.

This means my simulator demonstrates the security checks involved in a firmware update, but it does not complete the installation. A real-world IoT device would need additional mechanisms to securely write, install, and activate the new firmware.

Conclusion: My implementation demonstrates basic firmware security concepts, but it needs further development to represent a complete and secure firmware update process.

Functional Completeness Assessment (FCA)

```
PRIVATE_KEY.write_bytes(key.private_bytes(
    serialization.Encoding.PEM,
    serialization.PrivateFormat.PKCS8,
    serialization.NoEncryption()
))
```

Figure 17; FCA-1: Cryptographic Key Generation

I reviewed the initialise() function to check whether cryptographic key generation was implemented. The screenshot shows the simulator generating an Ed25519 private key and saving both the private and public keys (lines 26/28). This confirms that the required key generation functionality is present.

Result: Implemented.

```
signature = key.sign(canonical(message)).hex()
```

Figure 18; FCA-2: Firmware Digital Signing

I examined the build() function to determine whether firmware digital signing was implemented. The screenshot shows the private key being loaded and used to generate a digital signature for the firmware version and payload (line 49). This confirms that the simulator contains the required signing functionality.

Result: Implemented.

```
try:
    public_key.verify(
        bytes.fromhex(package["signature"]),
        canonical(message)
    )
```

Figure 19; FCA-3: Digital Signature Verification

FCA-03 — Digital Signature Verification

I reviewed the install() function to check whether signature verification was implemented. The screenshot shows the simulator loading a trusted public key and using it to verify the firmware's digital signature (line 74). It also includes logic to reject invalid signatures.

Result: Implemented.

```
message = {"version": version, "payload": payload}
signature = key.sign(canonical(message)).hex()
```

Figure 20; FCA-4: Firmware Integrity Protection

I reviewed how the firmware payload is included in the data used to generate and verify the digital signature in build(). The screenshot shows that both the firmware version and payload are signed. This confirms that the simulator includes an integrity verification mechanism designed to detect modifications to these fields.

Result: Implemented.

```
if message["version"] <= state["version"]:
    raise ValueError("Rejected: firmware downgrade or replay")
```

Figure 21; FCA-05: Firmware Downgrade Protection

I examined the install() function to check whether firmware downgrade protection was implemented. The screenshot shows a version comparison that rejects firmware with a version number equal to or lower than the currently recorded version.

Result: Implemented.

```
    )
except (InvalidSignature, ValueError, KeyError, TypeError):
    raise ValueError("Rejected: invalid firmware signature")
```

Figure 22; FCA-6: Invalid Signature Handling

I reviewed the exception-handling code in the install() function. The screenshot shows that the simulator rejects firmware packages if signature verification fails or the required signature field is missing.

Result: Implemented.

Identifying Missing Functions

Function	Status	Explanation
Actual firmware binary installation	Not implemented	Only the recorded version number is updated
Secure private-key storage	Not implemented	The private signing key is stored locally without encryption

There is also no hardware-backed protection for the recorded firmware version, so this simulation does not demonstrate resistance to an attacker who can modify the local device-state file.

Functional Sufficiency Assessment (FSA)

Objective

The aim of my Functional Sufficiency Assessment was to determine whether the security mechanisms implemented in my Python firmware simulator operated as intended.

I assessed the simulator's ability to accept valid firmware updates, detect unauthorised modifications, prevent firmware downgrades and reject unsigned firmware packages.

To evaluate these security functions, I reviewed the results of four tests (FW-1 to FW-4) and compared the actual outcomes against the expected results.

Test ID	Security Test	Expected Outcome	Actual outcome	Verdict
FSA-01	Valid signed update	Accepted	Version updated to 2	PASS
FSA-02	Tampered firmware	Rejected	Invalid signature error	PASS
FSA-03	Firmware downgrade	Rejected	Downgrade/reply error	PASS
FSA-04	Unassigned firmware	Rejected	Invalid signature error	PASS

FSA-1: Valid Firmware Update

I checked whether the simulator would accept a valid, digitally signed firmware update. The terminal confirmed that firmware version 2 was accepted, and the recorded version changed from 1 to 2.

Result: PASS: The mechanism operated as expected for a valid signed update.

Evidence: FW-01, page 4.

FSA-2: Tampered Firmware Detection

I checked whether the signature verification mechanism could detect unauthorised changes. After I modified the firmware payload without updating its digital signature, the simulator rejected the package and reported an invalid firmware signature.

Result: PASS: The simulator detected and rejected the modified package.

Evidence: FW-02, page 5.

FSA-3: Firmware Downgrade Protection

I tested whether the simulator would block installing an older firmware version. It rejected version 1 because version 2 was already recorded as installed.

Result: PASS: The version-checking mechanism prevented the downgrade under the tested conditions.

Evidence: FW-03, page 6.

FSA-4: Unsigned Firmware Rejection

I checked whether the simulator would reject a firmware package without a digital signature. The terminal reported an invalid signature, and the firmware version remained at 2.

Result: PASS: The simulator rejected the unsigned firmware package.

Evidence: FW-04, pages 6–7.

Identified Limitations

Although all four security tests passed, I identified limitations in the simulator:

- The simulator only updates the recorded firmware version; it doesn't install an actual firmware binary.
- The private signing key is stored locally without encryption.
- The recorded firmware version can be directly modified in the device-state file.
- The testing covers four selected scenarios and does not demonstrate protection against every possible attack.

These limitations mean a production IoT device would require additional security measures and testing.

Functional Sufficiency Assessment Conclusion

My Functional Sufficiency Assessment confirmed that the implemented security mechanisms performed as expected in all four test scenarios.

The simulator accepted a valid signed update and rejected tampered, outdated, or unsigned firmware packages.

However, the assessment relied solely on a Python-based simulation. As a result, the findings show how well the tested security functions performed in a laboratory setting, not whether they fully comply with ETSI EN 303 645.

Overall Findings and Recommendations

Throughout this assessment, I identified several limitations in my firmware update simulator.

Finding 1: Unencrypted Private-Key Storage

The private signing key is stored locally without encryption, which could expose it if an attacker gained access to the file.

Recommendation: Improve key protection through encrypted storage and appropriate access controls. For a production signing system, a dedicated hardware security module could provide stronger protection.

Finding 2: No Actual Firmware Installation

The simulator updates the recorded firmware version but does not install an actual firmware binary.

Recommendation: I would extend the simulator to securely write and validate firmware images before activation, including recovery procedures if installation fails.

Finding 3: Unprotected Device-State File

The installed firmware version is stored in a local JSON file that is not protected against direct modification.

Recommendation: I would introduce tamper-resistant version storage or an appropriate hardware-backed anti-rollback mechanism.

Finding 4: Limited Security Test Coverage

My testing covered four scenarios: valid firmware installation, tampered firmware, firmware downgrade and unsigned firmware rejection.

Recommendation: I would expand testing to include corrupted signatures, untrusted signing keys, malformed firmware packages and interrupted update processes.

Conclusion

For this project, I developed and tested a Python-based secure firmware update simulator to explore firmware authenticity, integrity, and update security. I implemented digital signature generation and verification, along with firmware version checks. I also carried out four security tests, and each produced the expected result.

I assessed the security design using Conceptual, Functional Completeness, and Functional Sufficiency Assessments, and reviewed and tested the behaviour of the implemented functions. The simulator demonstrated the intended security features under the conditions tested. However, I identified limitations in private-key storage, firmware installation, and device-state protection.

Overall, the project helped me build practical skills in cybersecurity testing, evidence gathering, security assessment, and technical reporting, while applying selected security principles from ETSI EN 303 645.

References

1. European Telecommunications Standards Institute (ETSI) (2020). *ETSI EN 303 645 V2.1.1: Cyber Security for Consumer Internet of Things: Baseline Requirements*. Available at: https://www.etsi.org/deliver/etsi_en/303600_303699/303645/02.01.01_60/en_303645v020101p.pdf (Accessed: 9 November 2026).

2. European Telecommunications Standards Institute (ETSI) (2025). *ETSI TS 103 701 V2.1.1: Cyber Security for Consumer Internet of Things: Conformance Assessment of Baseline Requirements*. Available at: https://www.etsi.org/deliver/etsi_ts/103700_103799/103701/02.01.01_60/ts_103701v020101p.pdf (Accessed: 9 November 2026).

3. Regenscheid, A. (2018). *Platform Firmware Resiliency Guidelines*. NIST Special Publication 800-193. National Institute of Standards and Technology. Available at: https://doi.org/10.6028/NIST.SP.800-193 (Accessed: 9 November 2026).

4. Department for Digital, Culture, Media and Sport (DCMS) (2018). *Code of Practice for Consumer IoT Security*. UK Government. Available at: https://www.gov.uk/government/publications/code-of-practice-for-consumer-iot-security (Accessed: 9 November 2026).

5. Josefsson, S. and Liusvaara, I. (2017). *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032. Internet Research Task Force. Available at: https://www.rfc-editor.org/rfc/rfc8032.html (Accessed: 9 November 2026).

6. Python Cryptographic Authority (PyCA) (n.d.). *Ed25519 Signing — Cryptography Documentation*. Available at: https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ed25519/ (Accessed: 9 November 2026).

7. Python Software Foundation (n.d.). *venv — Creation of Virtual Environments*. Python Documentation. Available at: https://docs.python.org/3/library/venv.html (Accessed: 9 November 2026).

8. pytest development team (n.d.). *pytest Documentation*. Available at: https://docs.pytest.org/en/stable/ (Accessed: 9 November 2026).